Lazy Periodic Rules: A Scalable Implementation of the Factor-2 Approximation for Shortest Common Superstring

Krushna Dere Svyatoslav Shmidt

October 2026

Abstract

The first polynomial-time factor-2 approximation for the shortest common superstring (SCS) problem was published in September 2026 as result 128 of the OpenAI mathematics collection, a non-peer-reviewed manuscript repository whose own catalogue warns that unformalized results "could have issues"; we therefore treat its proof as a hypothesis wherever we rely on it. Its lower bound is built from forced occurrence counts on every distinct substring of the input, including a periodic rule that applies to almost every word. Every implementation we know of therefore uses memory that grows quadratically in the input length. We show that the periodic rules can be enabled *lazily*, only for the periodic texts on which the construction actually invokes them, without losing the factor-2 guarantee: any subset of the rules still yields a valid lower bound $W' \leq \mathrm{OPT}$, and the construction uses a periodic rule at exactly one step, whose success is checked at run time. Without the periodic rules, the counts on each state of a generalised suffix automaton form a nonincreasing step function of the word length, which gives a representation with one step function per automaton state. The core of this argument is machine-checked in Lean 4 with Mathlib. We implement the algorithm in Rust together with an independent checker that re-justifies every count from scratch; it verifies exactly the hypothesis of the formalised certificate theorem. On error-free 150 bp reads at $10\times$ coverage of the *E. coli* K-12 genome (46.4 million input characters) the implementation runs in about two minutes on a laptop and returns a superstring within 0.021% of the certified lower bound. We also report branch coverage of the construction, the behaviour of greedy on Shibata's recent counterexample to the greedy conjecture, and a head-to-head comparison with two other published implementations.

1 Introduction

Given strings $s_1, \ldots, s_n$, the shortest common superstring problem asks for a shortest string containing every s_i as a contiguous substring. The problem is NP-hard and APX-hard, and a long line of approximation algorithms reached ratios below 2.5, the best published being $(\sqrt{67} + 14)/9 \approx 2.466$ [3], improving Mucha's $2\frac{11}{23}$ [5] and the 2.475 of Englert, Matsakis and Veselý (STOC 2022) [3]. The classical greedy algorithm was known to be at most a 3.396-approximation [3], improving the 3.5 of Kaplan and Shafrir [6] and the 4 of Blum et al. [4]. In September 2026 a polynomial-time factor-2 algorithm appeared as result 128 of the OpenAI mathematics collection [1] (henceforth *the paper*); it is a model-generated manuscript, not peer reviewed, and the repository itself states that not all results carry Lean formalizations and that "some of the unformalized results could have issues". Its statement is formalised in Lean in the same repository. Independently, Shibata [2] gave a preprint disproving the greedy superstring conjecture posed by Tarhio and Ukkonen in 1988 (nearly four decades earlier), showing that the classical greedy algorithm can be forced to a ratio approaching 9/4.

The paper's algorithm computes, for every distinct substring s of the input, a count $m(s)$ that no common superstring can undercut. Two kinds of rule define these counts: extension inequalities, and a periodic rule (2) indexed by a word v with a period $p < |v|$. Because every word with a border has such a period, the periodic rules range over almost all of the $\Theta(L^2)$ distinct substrings of an input of total length L. Within days of publication, implementations appeared in Lean, JavaScript, C++ and (partially) Rust [9, 10, 11, 12]; all of them enumerate distinct substrings explicitly.

On the practical side, Alanko and Norri [7] gave a compact-space implementation of the greedy heuristic, and Cazaux, Juhel and Rivals [8] computed fast two-sided bounds on the optimum; neither carries the worst-case factor-2 guarantee, which is what distinguishes the construction studied here.

Contributions. (i) We prove that the periodic rules may be enabled lazily, per periodic text, only when the construction needs them, while keeping the guarantee $|T| \le 2W' \le 2\,\mathrm{OPT}$ (Section 3). (ii) We show that the counts obtained without the periodic rules are piecewise constant on the states of a generalised suffix automaton, and give a representation with one nonincreasing step function per state, together with edge enumeration in $O(\text{states} + W)$ (Section 4). (iii) We provide a Rust implementation with an independent certificate checker (Section 5). (iv) We machine-check the core lower-bound argument in Lean 4. (v) We report correctness against exact optima, branch coverage, and scaling to a bacterial genome (Section 6).

2 Background

We follow the paper's notation and summarise only what we need. Let V be the set of distinct substrings of the (reduced) inputs, including the empty word ε. Counts $m(s)$ are defined in decreasing order of length as the least integers satisfying

$$m(s) \ge \sum_c m(cs), \qquad m(s) \ge \sum_c m(sc), \qquad m(s) \ge 1 \text{ if } s \text{ is an input}, \tag{1}$$

and, for every periodic text A of period p, every $v = A[0{:}|v|) \in V$ with $p < |v|$, and every prefix w of v with $|v| = |w| + kp$,

$$m(v) > B_A(v; m) \implies m(w) \ge B_A(w; m) + k + 1, \tag{2}$$

where $B_A(s;m) = \sum_r b_{A,s}(r)\, m(r)$ is the *blocking functional*: it counts occurrences of s that are bracketed, inside a word of V, by the nearest mismatches with A on both sides. The paper proves $m(s) \le N_T(s)$ for every common superstring T (Lemma 2.1), so $W = \sum_c m(c) \le \mathrm{OPT}$. Edge multiplicities $u(s) = m(s) - \sum_c m(cs)$ and $d(s) = m(s) - \sum_c m(sc)$ define a balanced "hierarchical" graph with W up edges. Its closed walks are reorganised into periodic *layers* (Section 3 of the paper), each of period p carrying a budget p; the budgets sum to W. Sections 4–6 of the paper connect every layer to ε with additional closed walks paid for by these budgets, and an Euler tour spells a common superstring of length at most $2W$.

3 Lazy periodic rules

The lazy variant computes counts m' from (1) exactly and from the rules (2) only for an explicit set E of enabled periodic texts (identified by period and least rotation). The enabled rules are applied as point lower bounds and iterated to a fixpoint: each pass evaluates every enabled rule on the current counts, raises $m'(w)$ where a triggered rule is violated, and recomputes the extension closure. The construction of the paper then runs unchanged. If its record search (Section 5.3 of the paper) fails, the text of the group being processed is added to E and the run restarts.

Lemma 1 (generalised Lemma 2.1)**.** *Let m' be any vector in which every value is the maximum of an extension bound, the required-string bound, and bounds from rules (2) whose triggers hold, all evaluated on values that do not exceed the true occurrence counts N_T. Then $m' \le N_T$ for every common superstring T; in particular $W' = \sum_c m'(c) \le$* OPT.

Proof. The paper's proof of Lemma 2.1 at a word w reads only counts of longer words and uses only that they are at most N_T. For a triggered rule there are two cases. If $N(v) > B_A(v;N)$, then $N(w) \ge B_A(w;N)+k+1 \ge B_A(w;m')+k+1$, because B_A has nonnegative coefficients and $m' \le N$ termwise. Otherwise $B_A(v;N) \ge N(v) \ge m'(v) > B_A(v;m')$, so some bracketing word r has $N(r) - m'(r) \ge 1$. Within r, the $k+1$ period-spaced prefixes of a counted placement of v give $b_{A,w}(r) \ge k+1$, hence $B_A(w;N) - B_A(w;m') \ge k+1$. Inducting over the sequence of assignments (rather than over word length) shows that every value ever assigned is at most N_T.

Theorem 2. *For every input, the lazy variant terminates in polynomial time and outputs a common superstring T with $|T| \le 2W' \le 2\,$*OPT.

Proof sketch. Lemma 2.2 of the paper (balance, W' up edges, required strings used) needs only (1) and the required-string bound, which the final pass enforces exactly. Lemmas 3.1–3.5 follow from the edge multiplicities and the definition of B_A. In particular, the trigger of the rule for $(A, v = A[t{:}H), w = A[t{:}H-K), k)$ in Section 5.3 holds automatically, because the top layer records an unblocked occurrence of v. Section 5.3 is the only step that invokes a periodic rule, and it does so only to show that a record exists. Every later step (Section 5.4, Lemmas 4.3–4.4, Section 6 of the paper) uses only properties of the record actually found, all of which are checked; the bound $b - a < p + q$ follows from Lemma 3.4 alone. If the search fails, enabling the group's text makes every triggered rule on it hold, so a text is never needed twice. There are at most $O(L^2)$ texts that can carry a rule, so the number of rounds is polynomial. Budgets sum to W' by Lemma 3.2, which gives $|T| \le 2W'$.

Formalisation. Lemma 1 is checked in Lean 4 (v4.35.0) with Mathlib, in the file `LazySCS.lean`, over an abstract type of words. Theorem `rule_sound` states that a periodic rule evaluated on any vector below the true counts is sound. Theorem `certificate` states that if every count is at most some sound bound that reads only strictly longer words, then every count is at most the true count. `lower_bound` sums this over one-letter words. The facts the paper proves about true occurrence counts enter as explicit hypotheses: its equation (2.4), the two cases of its Lemma 2.1, and the extension inequalities. Lean reports that all three theorems depend only on the standard axioms `propext`, `Classical.choice` and `Quot.sound`.

The bound $2W'$ is never weaker than the paper's $2W$, since $W' \le W$. The per-instance certificate $|T|/W'$ can be looser than $|T|/W$, but on every read set we tested no periodic rule was needed, so $W' = W$ there.

4 Counts on a suffix automaton

Let X be a state of the generalised suffix automaton of the inputs, holding the words of lengths $(\operatorname{len}(\operatorname{link} X), \operatorname{len} X]$. All words of X share their right extensions: sc lies in $\delta(X, c)$. Every word of X except the longest has exactly one left extension, namely the next longer word of X. The left extensions of the longest word are the shortest words of the states Z with $\operatorname{link} Z = X$. Hence, under (1) and the required-string bound alone, m restricted to X is the nonincreasing step function

$$m_X(\operatorname{len} X) = \max\Big(\sum_{\operatorname{link} Z = X} m_Z(\operatorname{minlen} Z),\ R_X(\operatorname{len} X),\ [X \ni \text{input}]\Big),$$
$$m_X(l) = \max\big(m_X(l+1), R_X(l)\big),$$

with $R_X(l) = \sum_c m_{\delta(X,c)}(l+1)$. Processing states by decreasing length respects every dependency. Enabled periodic rules enter as point lower bounds at (X, l). Edge multiplicities are nonzero only where m_X steps or where $m_X > R_X$; the latter ranges have total weight at most W, so all nonzero edges are enumerated in $O(\text{states} + W)$ time. A word is identified by the pair (state, length), and all walk operations (append a letter, delete the first letter) are $O(1)$ on these pairs. On the 4,503 test instances of Section 6, the compressed counts, W and every edge multiplicity agree exactly with an explicit per-substring reference implementation.

5 Implementation and certificate

The Rust implementation follows the paper's Sections 3–7 on (state, length) vertices. Each run checks at run time that the windows are vertices, that the walks are closed, that the budget is not exceeded, that $|T| \le 2W'$, and that every input occurs in T. Lemma 1 is the only ingredient of the guarantee that cannot be checked from the output, so we provide an **independent checker**. It recomputes, from the automaton and the final step functions alone, the extension sums, the suffix-link-tree sums and every enabled periodic rule, using its own straightforward implementation of B_A. It then confirms that each count is justified by a bound that mentions only strictly longer words. If every check passes, the formalised theorem `certificate` applies directly: its hypothesis is exactly what the checker verifies. Two implementation details mattered in practice. First, hashing (state, length) keys with

a multiplicative hash that mixes only towards the high bits made every bucket depend on the word length alone, which produced quadratic behaviour; a splitmix64 finaliser fixed it. Second, the blocking functional must extend matches one letter at a time rather than re-walking from the root.

6 Experiments

All experiments ran on a Windows laptop with 24 GB of RAM. Read sets are error-free 150 bp reads sampled uniformly at 10× coverage.

6.1 Correctness

We use 4,503 instances from six families: mixed random, random over four letters, genome-like reads, periodic, nested periodic, and Shibata instances. On every instance, both the Python and the Rust implementations of the lazy variant return a valid superstring with $|T| \leq 2W'$, and $W' \leq \mathrm{OPT}$ whenever the exact optimum is computable (by subset dynamic programming, up to 12 strings). The worst observed ratio is 1.96. The independent checker justified all 402,535 checkpoints with no failures. A further 160,000 random repetitive instances (Section efsubsec:branch) also passed every run-time check.

Family	n	rules	enabled mean/max	no rule	rounds
mixed random	3000	150	1.39 / 6	30%	7
random, 4 letters	400	74	0.01 / 1	98%	2
genome-like reads	400	423	0.00 / 0	100%	1
periodic	400	350	2.92 / 6	1%	7
nested periodic	300	435	2.25 / 6	6%	7

Table 1: Lazy rules versus the full rule set (average number of rules per instance; "rounds" is the maximum number of restarts). No static subset of rules is safe: with only "square" rules ($|v| \geq 2p$), 189 of 4,503 instances fail at Section 5.3 of the paper.

6.2 Branch coverage

Randomised testing almost never reaches the deeper branches of the construction, so we searched specifically for inputs that trigger them, over 100,000 random repetitive instances plus 60,000 with alternative record-search orders. Host requests (Lemma 4.4 of the paper) fired 213 times; the smallest trigger is the two-string input `aaaa`, `aaabaaaaaabaaaaaa`. Opened cycles fired 828 times, band joins 723, short-contact links 108, upper round trips 19 and individual links 2. Every trigger also passed the exact-optimum check. Two branches never fired: layers rooted below a baseline (Section 5.1 of the paper) and single-block cycles (Section 6 of the paper). We report them as untested.

6.3 Quality versus greedy

6.4 Scale and comparison with other implementations

We ran our implementation and the two other complete, runnable implementations, JavaScript [10] and C++ [11], on error-free 150 bp reads at 10× coverage from prefixes

Family (300 inst.)	greedy m/w	2-apx raw	2-apx trim	best both
random, 4 letters	1.005 / 1.074	1.252 / 1.688	1.103 / 1.500	1.004 / 1.074
random, 2 letters	1.012 / 1.136	1.265 / 1.867	1.082 / 1.462	1.007 / 1.133
genome reads	1.000 / 1.026	1.262 / 1.643	1.054 / 1.552	1.000 / 1.026
periodic	1.001 / 1.091	1.441 / 1.939	1.106 / 1.692	1.001 / 1.091

Table 2: Length divided by the exact optimum on small instances (m/w = mean/worst). On typical inputs greedy is better than the factor-2 construction; the guarantee matters only in the worst case.

k, t	OPT	greedy adv. ties	ratio	2-apx	ratio
10, 5	119	231	1.941	215	1.807
10, 20	449	921	2.051	860	1.915
14, 10	313	641	2.048	590	1.885
20, 10	439	911	2.075	830	1.891

Table 3: Shibata's instances $S_{k,t}$ [2]. We replay the greedy run from [2] and check that every merge has the current maximum overlap. Greedy exceeds twice the optimum, while the factor-2 construction stays below it. With our own deterministic tie-breaking, greedy happens to avoid the trap (e.g. 685 for $k = 10, t = 20$).

of the *E. coli* K-12 genome (NC_000913.3). Each run had a limit of 300 s. The Lean implementation [9] reports timeouts on five strings of three letters, so we did not include it. We built the C++ code in release mode with MSVC, after replacing its GCC-only warning flags.

7 Limitations

The lower-bound half of Theorem 2 is machine-checked, but only relative to the paper's facts about true occurrence counts, which we take as hypotheses rather than re-deriving from strings. The other half, that the construction invokes a periodic rule only at its Section 5.3, is an informal argument from re-reading the paper. It is supported by run-time checks of every step and by the fact that no requested text was ever requested twice. The independent checker and the Rust implementation are themselves unverified code. Two branches of the construction were never exercised (Section efsubsec:branch). The read sets are error-free; real sequencing errors are outside the SCS model. On typical inputs, greedy produces shorter superstrings, so the contribution here is scale and certification, not better average output. Finally, both [1] and [2] are unrefereed preprints: the status of [1] in particular may change as the collection is revised, and our results are conditional on its lemmas about true occurrence counts, which we verify only as hypotheses in Lean.

Disclosure of AI assistance

An AI assistant (referred to here as wigip) wrote the implementation, ran the experiments, and drafted this manuscript under the authors' direction. The authors directed the work, chose its scope and are responsible for the content. The lazy-rule argument (Section 3) was proposed and checked with the assistant's help and has not been independently peer reviewed.

Prefix	chars	JS [10]	C++ [11]	this work	$\lvert T\rvert/W$
200 bp	1,950	0.12 s	111 s	<0.01 s	1.000
1 kb	9,900	0.87 s, 261 MB	timeout	0.08 s, 2 MB	1.130
3 kb	30,000	2.2 s, 491 MB	—	0.12 s, 6 MB	1.047
10 kb	99,900	7.3 s, 1.2 GB	—	0.22 s, 16 MB	1.000
30 kb	0.3 M	24 s, 3.4 GB	—	0.52 s, 44 MB	1.000
100 kb	1.0 M	timeout (10.9 GB)	—	1.9 s, 143 MB	1.00001
300 kb	3.0 M	—	—	6.2 s, 417 MB	1.00095
1 Mb	10.0 M	—	—	22 s, 1.4 GB	1.00042
4.64 Mb (full)	46.4 M	—	—	121 s, 6.3 GB	1.00021

Table 4: Wall time and peak memory, including our end trimming; "—" means the implementation was not run after an earlier timeout. Wherever both completed, our output length equals the JavaScript output length exactly (e.g. 29,954 at 30 kb). For the full genome, $W = 4{,}602{,}085$ and $|T| = 4{,}603{,}069$; the construction uses 12 periodic groups and needs no periodic rule. The independent checker verifies all 80.6 million checkpoints in about 14 s more. The superstring is shorter than the genome because SCS merges repeats longer than the read length. This is a property of the problem, and it is why genome assemblers do not use SCS.

Code and data availability

Source code, Python package, Lean formalisation, test generators, trigger instances and benchmark scripts: https://github.com/nurric-ai/lazy-scs, archived as doi:10.5281/zenodo.23362570 (version 1.0.0: doi:10.5281/zenodo.23362571); Python package: `pip install lazy-scs`. The *E. coli* reference is NCBI NC_000913.3.

References

[1] OpenAI. A polynomial-time 2-approximation for shortest common superstring. Result 128, *openai/math* manuscript collection, September 24, 2026. https://github.com/openai/math

[2] H. Shibata. Disproving the greedy superstring conjecture. arXiv:2609.01365, 2026.

[3] M. Englert, N. Matsakis, P. Veselý. Approximation guarantees for shortest superstrings: simpler and better. ISAAC 2023.

[4] A. Blum, T. Jiang, M. Li, J. Tromp, M. Yannakakis. Linear approximation of shortest superstrings. J. ACM 41(4), 1994.

[5] M. Mucha. Lyndon words and short superstrings. SODA 2013.

[6] H. Kaplan, N. Shafrir. The greedy algorithm for shortest common superstrings. Inf. Process. Lett. 93(1), 2005.

[7] J. Alanko, T. Norri. Greedy shortest common superstring approximation in compact space. arXiv:1707.07727, 2017.

[8] B. Cazaux, S. Juhel, É. Rivals. Practical lower and upper bounds for the shortest linear superstring. SEA 2018.

[9] buberlo/veriscs: executable machine-checked 2-approximation (Lean 4). GitHub, 2026.

[10] V. Karakoidas. shortest-superstring-js. GitHub, 2026.

[11] spele1100/scs-2approx: certified factor-2 approximation (C++17). GitHub, 2026.

[12] rongou/superstring-rs (Rust, partial). GitHub, 2026.